

Apprendre à justifier

On cherche ici à justifier un texte sur la largeur. On suppose que l'on dispose d'un objet de type `string list` qui donne la liste des mots du texte. On dispose de la largeur notée `lg` dans la suite des lignes de la zone d'affichage. On cherche à afficher le texte de manière harmonieuse, en alignant les mots sur les bords gauche et droit de la zone d'affichage.

Par exemple, considérons le texte représenté par la liste de mots définie par:

```
let debarquement = ["Les"; "sanglots"; "longs"; "des"; "violons"; "de"; "l'automne"; "blessent\"
  "; "mon"; "coeur"; "d'une"; "langueur"; "monotone."; "Tout"; "suffocant"; "et"; "bleme\"
  "; "quand"; "sonne"; "l'heure, "; "je"; "me"; "souviens"; "des"; "jours"; "anciens"; "et\"
  "; "je"; "pleure"; "et"; "je"; "m'en"; "vais"; "au"; "vent"; "mauvais"; "qui"; "m'emporte\"
  "; "deca"; "dela"; "pareil"; "a"; "la"; "feuille"; "morte."];;
```

On aimerait alors afficher, pour une largeur de ligne fixée à trente caractères:

```
Les      sanglots      longs      des
violons  de l'automne blessent
mon      coeur      d'une  langueur
monotone. Tout suffocant et
bleme    quand sonne l'heure,
je      me souviens des jours
anciens et je pleure et je
m'en    vais au vent mauvais
qui     m'emporte deca dela
pareil  a la feuille morte.
```

Notons que l'on ne gère pas ici les cédilles, lettres accentuées et autres caractères spéciaux. De même, on note que tout symbole de ponctuation appartient au mot qui le précède.

I Préliminaire d'affichage

1) Coder la fonction `ligne_to_affichable (lg: int) (ligne: string list): string`. Cette fonction prend une liste de mots `ligne` représentant les mots d'une ligne et renvoie une chaîne de caractère permettant d'afficher de manière justifiée sur la largeur cette liste de mots dans une ligne de longueur `lg`. Le résultat sera donc un `string` de longueur `lg` qui ne finira pas par un espace. On supposera que la largeur `lg` est suffisante.

Par exemple, `ligne_to_affichable 15 ["ut"; "enim"; "ad"];;` renvoie:

```
"ut      enim      ad"
```

2) Coder une fonction d'affichage d'une liste de string, qui affiche chacun des éléments de la liste en les séparant par un saut de ligne. Elle sera d'en-tête: `affiche (liste_string: string list):unit`

II Approche gloutonne

Dans un premier temps, on choisit de procéder de manière gloutonne: on met dans la première ligne autant de mots de mots que possible, puis dans la seconde autant de mots non encore affichés que possible et ainsi de suite pour chacune des autres lignes.

3) Coder la fonction `glouton (lg:int) (liste_mots: string list): string list list` qui prend en argument la longueur `lg` des lignes, la liste des mots à afficher et renvoie une liste de listes, chaque sous-liste représentant la liste des mots à afficher sur une ligne du texte.

Par exemple, en définissant: `let essai1 = ["ut"; "enim"; "ad"; "minima"; "veniam"];;`

L'appel de `glouton 10 essai1` renvoie le `string list list`:

```
[["ut"; "enim"; "ad"]; ["minima"]; ["veniam"]].
```

4) En déduire la fonction `affichage_glouton (lg:int) (liste_mots: string list):unit` qui affiche le texte représenté par la liste de mots dans une succession de lignes de même largeur `lg`. Par exemple, l'appel de `affichage_glouton 10 essai1` donnera

```
ut enim ad
  minima
  veniam
```

Tandis que `affichage_glouton 30 debarquement` affichera:

```
Les sanglots longs des violons
de l'automne blessent mon
coeur d'une langueur monotone.
Tout suffocant et bleme quand
sonne l'heure, je me souviens
des jours anciens et je pleure
et je m'en vais au vent
mauvais qui m'emporte deca
dela pareil a la feuille
                morte.
```

Le résultat est donc relativement peu satisfaisant, en particulier, à cause de la dernière ligne réduite à un mot.

Pour évaluer la pertinence du placement d'espaces et de retour à la lignes, on définit une fonction coût qui représente le degré de mocheté d'un choix d'affichage. On cherchera à minimiser ce coût.

Le coût d'une ligne est égal au carré du nombre d'espaces à ajouter (en sus des espaces obligatoires entre chaque mot) pour obtenir une ligne de taille voulue à partir des mots affectés à cette ligne. Si la ligne est trop courte pour afficher les mots choisis, le coût sera considéré comme égal à l'infini.

Le coût total est la somme des coûts de chaque ligne.

5) Observer que la méthode glouton ne donne pas le coût optimal dans l'exemple de `essai1`.

III Programmation dynamique

6) Coder la fonction `fabric_tab_lg (liste_mots: string list): int array` qui permet de passer de passer d'une liste de mots au tableau des longueurs de ces mots. Par exemple, `fabric_tab_lg essai1` renverra `[12; 4; 2; 6; 6]`.

7) Coder la fonction de calcul du coût d'affichage sur une même ligne de longueur `lg` quand on sélectionne les mots du mot d'indice `i` à celui d'indice `j`:

```
cout (i:int) (j:int) (tab_lg: int array) (lg: int).
```

On note dans la suite $d(i)$ le coût d'affichage optimal de tous les mots à partir de celui d'indice i inclus.

8) Trouver une formule de récurrence sur les $d(i)$.

9) En déduire une fonction très naïve `calcul_d_min (i:int) (tab_lg:int array) (lg:int): int` qui prend en argument un indice entier i , le tableau des longueurs des mots et renvoie la valeur de $d(i)$ lorsque la largeur des lignes est égale à `lg`.

Par exemple, `calcul_d_min 0 lg_mots1 10` renvoie 26. (On désigne ici par `lg_mots1` le tableau des longueurs des mots correspondants à l'exemple de `essai1`).

Quelle est la complexité de cette fonction?

10) Améliorer cette fonction en mémorisant les résultats obtenus dans une table de hachage, on définira donc: `calcul_d_min_memo (k:int) (tab_lg:int array) (lg:int):int`.

11) On cherche maintenant à donner une version itérative de la fonction précédente. Remarquons qu'il nous suffit de calculer le tableau des $d(i)$ pour $i \in \llbracket 0, n \rrbracket$ où n est le nombre de mots du texte à écrire.

a) Dans quel ordre obtient-on les différentes valeurs de $d(i)$?

b) En déduire une fonction de calcul du couple `tab_cout`, `tab_decoupe`, où `tab_cout` est le tableau des $d(i)$ et `tab_decoupe` est le tableau qui code les découpages de lignes optimaux à effectuer. Plus précisément, pour afficher harmonieusement tout le texte à partir du mot d'indice i , on doit commencer par une première ligne qui comprend les mots d'indice i inclus à `tab_decoupe.(i)` exclus, la deuxième ligne commençant avec le mot d'indice `tab_decoupe.(i)`. La fonction sera d'en-tête:

```
calcul_cout_PD (tab_lg: int array) (lg:int) et de type
calcul_cout_PD : int array -> int -> int array * int array.
```

12) En déduire une fonction `affichage_PD (lg:int) (liste_mots:string list):unit` d'affichage de manière justifiée et harmonieuse du texte défini par la liste de mots `liste_mots` lorsque la largeur choisie est `lg`.

Par exemple, `affichage_PD 10 essai1` permet d'afficher:

```
ut    enim
ad    minima
      veniam
```

Par exemple, `affichage_PD 30 debarquement` affiche le texte donné en préambule.